

5_Widgets

March 3, 2022

1 Widgets

Są to obiekty w Pythonie reagujące na zdarzenia i umożliwiające obsługę różnych popularnych kontrolek w przeglądarce. Wykorzystane zostaną do utworzenia prostego GUI do obsługi robota.

```
[2]: import ipywidgets as widgets
```

```
[2]: import rospy
      from geometry_msgs.msg import Twist
```

```
[ ]: rospy.init_node("widgets_controller")
```

Unable to register with master node [http://localhost:11311/]: master may not be running yet. Will keep trying.

Na zajęciach w których omawiany był Publisher wysyłanie prędkości sterujących odbywało się w następujący sposób:

```
[ ]: twist_publisher= rospy.Publisher("/turtle1/cmd_vel",Twist,queue_size=10)

      some_message=Twist()
      some_message.angular.z=1
      some_message.linear.x=10

      twist_publisher.publish(some_message)
```

```
[ ]: def move_robot(forward_vel=5,rotation_vel=5):
      '''A function to move turtle from turtlesim simulation'''

      Args:
      forward_vel (float): Linear velocity
      rotation_vel (float): Angular velocity'''
      message=Twist()
      message.angular.z=rotation_vel
      message.linear.x=forward_vel

      twist_publisher.publish(message)
```

```
[ ]: move_robot(1,-1)
```

2 Slider

Jak widać zmiana prędkości odbywa się poprzez ręczne wprowadzenie wartości i wywołanie funkcji. Z punktu widzenia użytkownika oczekiwałby on, aby mógł sterować robotem przy pomocy kontrolerek. Wykorzystamy funkcję **move_robot** do utworzenia domyślnego widgetu sterującego prędkościami robota. Każda zmiana wartości kontrolki powoduje wywołanie funkcji **move_robot** i wysłanie odczytanych prędkości z kontrolerek.

```
[ ]: widgets.interact(move_robot)
```

Można również zdefiniować własne kontrolki. Poniżej przykład z użyciem **FloatSlider** dla, którego ustawiane są wartości minimalne, maksymalne, krok zmiany oraz wartość początkowa. Jako pierwszy argument podawana jest nazwa funkcji, która ma zostać wywołana. Dla argumentów jakie przyjmuje funkcja *move_turtle* utworzone zostały odpowiednie slidery.

```
[ ]: widgets.interact(move_robot,
                      forward_vel=widgets.FloatSlider(min=-10,max=10,step=2,value=0),
                      rotation_vel=widgets.FloatSlider(min=-3,max=3,value=0))
```

Zadanie 1 Utworzyć slidery do zmiany koloru tła 1. Zaimportować odpowiednie biblioteki dla sersicu /clear 2. Utworzyć funkcję *set_color* przyjmującą 3 argumenty koloru, która ustawia odpowiednie wartości parametrów koloru tła. Było na zajęciach z ROS Parameter Server. 3. Wyczyścić serwisem /clear tło mapy, aby możliwa była aktualizacja koloru. 4. Utworzyć widget z 3 sliderami ustawiającymi kolor.

```
[ ]: # UZUPEŁNIĆ
```

```
[ ]:
```

3 Textbox

Teraz czas na wysłanie dowolnej informacji na topicu **/informacja**. Skorzystanie z kontrolki do wprowadzania tekstu spowoduje, że każde wprowadzenie nowego znaku wyśle wiadomość. W terminalu można podejrzeć informację na tym topicu.

rostopic echo /informacja

Oto przykład:

```
[ ]: from std_msgs.msg import String
def send_msg(wiadomosc):
    pub_info = pub_speed=rospy.Publisher("/informacja",String,queue_size=10)
    msg_info = String()
```

```
msg_info.data = wiadomosc
pub_info.publish(msg_info)
```

```
[ ]: widgets.interact(send_msg,
                        wiadomosc=widgets.Text(value='Hello World!'))
```

4 Button

Jeśli chcielibyśmy wysłać dowolną informację po zakończeniu jej wprowadzania przydałby się przycisk *wyślij* odpowiedzialny za przekazanie informacji.

```
[ ]: widgets.ToggleButton(
    value=False,
    description='Click me',
    disabled=False,
    button_style='', # 'success', 'info', 'warning', 'danger' or ''
    tooltip='Description',
    icon='check' # (FontAwesome names without the `fa-` prefix)
)
```

Teraz czas na modyfikację kodu do wysyłania informacji po wciśnięciu przycisku *wyślij*. W tym celu tworzona jest zmienna *input_text*. Do wyświetlenia kontrolki służy funkcja *display()*.

```
[ ]: input_text = widgets.Text(value='Hello World!', disabled=False,
                               ↳description="informacja:")
display(input_text)
```

Odbiór wartości z kontrolki.

```
[ ]: odczytany_tekst = input_text.value
print(odczytany_tekst)
print("typ wartości: ", type(odczytany_tekst))
```

Jak można zauważyć pomimo tego, że tekst wprowadzamy jako string to po odczytaniu wartości jest ona typu 'unicode'. Jeśli mamy wysłać wiadomość tekstową to należy dokonać konwersji typu na str.

```
[ ]: odczytany_tekst = str(input_text.value)
print(odczytany_tekst)
print("typ wartości: ", type(odczytany_tekst))
```

Funkcja *send_msg* jako argument przyjmuje informację o stanie przycisku *wyślij*, z którego pochodzi żądanie wywołania tej funkcji.

```
[ ]: from std_msgs.msg import String
def send_msg(button_data):
    pub_info = pub_speed=rospy.Publisher("/informacja",String,queue_size=10)
```

```
msg_info = String()
msg_info.data = str(input_text.value)
pub_info.publish(msg_info)
```

Utworzenie przycisku do wysyłania wiadomości.

```
[ ]: przycisk_wyslij = widgets.Button(
    value=False,
    description='wyslij',
    disabled=False,
    button_style='', # 'success', 'info', 'warning', 'danger' or ''
    tooltip='Description',
    icon='check' # (FontAwesome names without the `fa-` prefix)
)
display(przycisk_wyslij)
```

Obsługa zdarzenia przycisku na kliknięcie. Po wciśnięciu przycisku wyślij następuje wywołanie funkcji `send_msg` podanej jako argument w poniższej metodzie dla zmiennej przechowującej informację o obiekcie przycisku. Wywołana funkcja jako argument przyjmuje informację o stanie przycisku, z którego pochodzi sygnał.

```
[ ]: przycisk_wyslij.on_click(send_msg)
```

Zadanie 2 Utworzyć publisher'a i slidery do wysyłania prędkości postępowej i obrotowej do robota. Można skorzystać z dostępnych sliderów na początku tego skryptu. Wiadomość o prędkościach powinna pojawiać się na topicu `/nazwa_robota/cmd_vel/slider`.

```
[ ]: # UZUPEŁNIĆ
```

Zadanie 3 Utworzyć dwie kontrolki do wprowadzania prędkości. Wysłanie prędkości powinno zostać zatwierdzone przez wciśnięcie przycisku. Można skorzystać z kontrolki `FloatText` przedstawionej poniżej. Można zauważyć, że zwracana wartość przez kontrolkę jest typu float i nie trzeba dokonać konwersji danych.

```
[ ]: liczba = widgets.FloatText(
    value=7.5,
    description='Any:',
    disabled=False
)
display(liczba)
odczytana_liczba = liczba.value
print(odczytana_liczba)
print("typ wartości: ", type(odczytana_liczba))
```

```
[ ]: # UZUPEŁNIĆ
```

Zadanie 4 Utworzyć przycisk do wysyłania prędkości wraz z funkcją, która będzie wysyłała wartość z przycisków na topic `/nazwa_robota/cmd_vel/float_text`.

```
[ ]: # UZUPEŁNIĆ
```

```
[ ]:
```

Zadanie 5 Utworzyć parametr `select_controler` oraz uzupełnić poniższą klasę.

```
[ ]: rospy.set_param(..., ...) # UZUPEŁNIĆ ustawić parametr velocity_source, 1 -  
    ↪ źródło slidery, 2 - float_text  
class VelocityControler:  
    def __init__(self):  
        slider_sub = ... # UZUPEŁNIĆ subscriberem od topicu ze sliderów, do  
        ↪ odczytu wykorzystać  
        # metodę klasy callback_slider. Odwołanie się do metod  
        ↪ klasy następuje poprzez self.nazwa_metody w klasie.  
        float_text_sub = ... # UZUPEŁNIĆ subscriberem od wartości prędkości  
        ↪ wprowadzanych z pól tekstowych callback_float_text  
        self.vel_pub = ... # UZUPEŁNIĆ utworzyć publisher, który będzie wysyłał  
        ↪ wiadomości w zależności od ustawionego  
        # parametru velocity_source do robota. Gdzie wartość  
        ↪ parametru 1 oznacza, że źródłem jest  
        # slider, a wartość 2 - float_text. Dla pozostałych  
        ↪ wartości parametru robot stoi w miejscu.  
        # Wysyłana jest prędkość 0 zatrzymująca robota. Wiadomość  
        ↪ powinna zostać opublikowana na topic  
        # sterującym robotem /nazwa_robota/cmd_vel  
  
    def callback_slider(self, msg_data):  
        # UZUPEŁNIĆ zweryfikować stan parametru i jeśli tryb jest właściwy  
        ↪ przekazywać prędkości sterujące  
        ...  
  
    def callback_float_text(self, msg_data):  
        # UZUPEŁNIĆ zweryfikować stan parametru i jeśli tryb jest właściwy  
        ↪ przekazywać prędkości sterujące  
        ...
```

```
[ ]: VelocityControler()
```

5 Checkbox

Przykład użycia

```
[ ]: widgets.Checkbox(
    value=False,
    description='Check me',
    disabled=False,
    indent=False
)
```

Zadanie 6 Odczytać wartość z przycisku i ustawić parametr **velocity_source**.

```
[ ]: # UZUPEŁNIĆ

def set_velocity_source(slider_source):
    #UZUPEŁNIĆ
    ...

widgets.interact(set_velocity_source,
                  slider_source = ... # UZUPEŁNIĆ
                  )
```

6 Tabs

Do grupowania zakładek i wyświetlania wszystkich w jednej karcie służy **VBox** z omawianej biblioteki. Przyjmuje listę kontrolki do wyświetlenia.

```
[3]: tab_contents = ['P0', 'P1', 'P2']
children = [widgets.VBox([widgets.Text(description=name), widgets.IntSlider(),
    ↪widgets.IntSlider(), widgets.IntSlider()]) for name in tab_contents]
tab = widgets.Tab(children = children)
for i in range(len(tab_contents)):
    tab.set_title(i, tab_contents[i])
tab
```

Tab(children=(VBox(children=(Text(value='', description='P0'), IntSlider(value=0), IntSlider(va

```
[4]: kontrolka1 = widgets.IntSlider(description="speed")
kontrolka1_2 = widgets.Checkbox(description="slider_source")
kontrolka2 = widgets.IntSlider(description="cos tam")
kontrolka3 = widgets.Text(description="info")
kontrolka3_2 = widgets.IntSlider(description="forward")
kontrolka3_3 = widgets.IntSlider(description="rotational")

children = [
    widgets.VBox([kontrolka1, kontrolka1_2]),
```

```

        widgets.VBox([kontrolka2]),
        widgets.VBox([kontrolka3, kontrolka3_2, kontrolka3_3])
    ]

    tab = widgets.Tab(children = children)
    # ustawianie nazwy zakładek; Kolejne argumenty: id zakładki, nazwa zakładki
    tab.set_title(0, "P1")
    tab.set_title(1, "P2")
    tab.set_title(2, "P3")
    tab

```

```

Tab(children=(VBox(children=(IntSlider(value=0, description='speed'), Checkbox(value=False, des

```

6.0.1 Dodatkowe widgety

<https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20List.html> ### Zmiana stylu i układu kontrolki <https://ipywidgets.readthedocs.io/en/latest/examples/Widget%20Styling.html>

[]: